Binary Search Tree Data Structure

1. Traversal in BST

What is Tree Traversal?

Tree traversal is the process of visiting each node in a tree data structure exactly once. In Binary Search Trees, there are three main types of traversal methods:

Types of BST Traversal

1.1 Inorder Traversal (Left → Root → Right)

In inorder traversal, we visit nodes in the following order:

- First, visit the left subtree
- Then, visit the root node
- Finally, visit the right subtree

```java
public void inorderTraversal(TreeNode root) {
    if (root != null) {
        inorderTraversal(root.left);   // Visit left subtree
        System.out.print(root.data + " "); // Visit root
        inorderTraversal(root.right);  // Visit right subtree
    }
}
```

1.2 Preorder Traversal (Root → Left → Right)

In preorder traversal, we visit nodes in the following order:

- First, visit the root node
- Then, visit the left subtree
- Finally, visit the right subtree

```java
public void preorderTraversal(TreeNode root) {
    if (root != null) {
        System.out.print(root.data + " "); // Visit root
        preorderTraversal(root.left);   // Visit left subtree
        preorderTraversal(root.right);  // Visit right subtree
    }
}
```

1.3 Postorder Traversal (Left → Right → Root)

In postorder traversal, we visit nodes in the following order:

- First, visit the left subtree
- Then, visit the right subtree
- Finally, visit the root node

```java
public void postorderTraversal(TreeNode root) {
    if (root != null) {
        postorderTraversal(root.left);  // Visit left subtree
        postorderTraversal(root.right); // Visit right subtree
        System.out.print(root.data + " "); // Visit root
    }
}
```

Basic TreeNode Class Structure

```java
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Example BST and Traversal Results

Consider this BST:

```
        50
       /  \
      30   70
     / \   / \
    20 40 60 80
```

- **Inorder**: 20, 30, 40, 50, 60, 70, 80
- **Preorder**: 50, 30, 20, 40, 70, 60, 80
- **Postorder**: 20, 40, 30, 60, 80, 70, 50

2. Importance of Inorder Traversal in BST

Why is Inorder Traversal Special in BST?

2.1 Sorted Order Property

The most important characteristic of inorder traversal in a BST is that it visits all nodes in **sorted ascending order**. This happens because of the BST property:

- All nodes in the left subtree are smaller than the root
- All nodes in the right subtree are greater than the root

2.2 Practical Applications

Retrieving Sorted Data

```java
public void printSortedElements(TreeNode root) {
    // This will print all elements in ascending order
    inorderTraversal(root);
}
```

Finding Kth Smallest Element

```java
class Solution {
    private int count = 0;
    private int result = -1;

    public int kthSmallest(TreeNode root, int k) {
        inorderHelper(root, k);
        return result;
    }

    private void inorderHelper(TreeNode root, int k) {
        if (root == null || count >= k) return;

        inorderHelper(root.left, k);
        count++;
        if (count == k) {
            result = root.data;
            return;
        }
        inorderHelper(root.right, k);
    }
}
```

Validating if a Tree is a Valid BST

```java
public boolean isValidBST(TreeNode root) {
    List<Integer> inorderList = new ArrayList<>();
    inorderTraversal(root, inorderList);

    // Check if the inorder traversal is sorted
    for (int i = 1; i < inorderList.size(); i++) {
        if (inorderList.get(i) <= inorderList.get(i - 1)) {
            return false;
        }
    }
    return true;
}

private void inorderTraversal(TreeNode root, List<Integer> list) {
    if (root != null) {
        inorderTraversal(root.left, list);
        list.add(root.data);
        inorderTraversal(root.right, list);
    }
}
```

2.3 Key Benefits of Inorder Traversal in BST

1. **Automatic Sorting**: No need for external sorting algorithms
2. **Space Efficient**: Can process elements one by one without storing entire sorted array
3. **Range Queries**: Easy to find elements within a specific range
4. **Tree Validation**: Quick way to check if BST property is maintained

Range Query Example

```java
public void printRange(TreeNode root, int min, int max) {
    if (root == null) return;

    // Only traverse left if current node is greater than min
    if (root.data > min) {
        printRange(root.left, min, max);
    }

    // Print current node if it's within range
    if (root.data >= min && root.data <= max) {
        System.out.print(root.data + " ");
    }

    // Only traverse right if current node is less than max
    if (root.data < max) {
        printRange(root.right, min, max);
    }
}
```

Quiz Questions

1. What is the correct order of node visits in inorder traversal? a) Root → Left → Right b) Left → Root → Right c) Left → Right → Root d) Right → Root → Left

Answer: b) Left → Root → Right

2. For the BST with root 15, left child 10, and right child 20, what is the inorder traversal result? a) 15, 10, 20 b) 10, 20, 15 c) 10, 15, 20 d) 20, 15, 10

Answer: c) 10, 15, 20

3. Why is inorder traversal particularly important in Binary Search Trees? a) It visits nodes in random order b) It visits nodes in sorted ascending order c) It is the fastest traversal method d) It uses the least memory

Answer: b) It visits nodes in sorted ascending order

4. In a BST, if you want to find the kth smallest element, which traversal method is most efficient? a) Preorder traversal b) Postorder traversal c) Inorder traversal d) Level order traversal

Answer: c) Inorder traversal

5. What would be the inorder traversal of a BST with values: 25 (root), 15 (left child of 25), 35 (right child of 25), 10 (left child of 15), 22 (right child of 15)? a) 25, 15, 10, 22, 35 b) 10, 15, 22, 25, 35 c) 10, 22, 15, 35, 25 d) 25, 15, 35, 10, 22

Answer: b) 10, 15, 22, 25, 35

Practice Coding Questions

1. Write a Java method to perform inorder traversal of a BST and store the result in an ArrayList. The method should return the ArrayList containing all node values in sorted order.

2. Implement a method that uses inorder traversal to find the second largest element in a Binary Search Tree. Handle the case where the tree has fewer than 2 nodes.

3. Create a method that performs inorder traversal and prints only the leaf nodes of a BST in ascending order. A leaf node is a node with no left or right children.

4. Write a Java method that uses inorder traversal to count how many nodes in a BST fall within a given range [min, max] (inclusive). The method should be efficient and avoid visiting unnecessary nodes.

5. Implement a method that performs inorder traversal to check if two BSTs contain the same elements (regardless of structure). Return true if both trees contain identical sets of values, false otherwise.